

3. Le dépôt en mémoire (Repository)

Étape 1 — EILMS : Domain Model & In-Memory Data Layer

Fichier concerné :

src/main/java/com/eilms/repository/InMemoryInventoryRepository.java

C'est la couche de persistance en mémoire, thread-safe, qui gère les données d'inventaire lors d'accès concurrents (par exemple plusieurs requêtes web arrivant en même temps).

```
public class InMemoryInventoryRepository {

    // ConcurrentHashMap garantit un accès thread-safe et performant
    private final Map<String, WarehouseItem> storage = new ConcurrentHashMap<>();

    public void save(WarehouseItem item) {
        if (item == null || item.getId() == null) {
            throw new IllegalArgumentException("Item and Item ID must not be null.");
        }
        storage.put(item.getId(), item);
    }

    public Optional<WarehouseItem> findById(String id) {
        return Optional.ofNullable(storage.get(id));
    }

    public List<WarehouseItem> findAll() {
        return new ArrayList<>(storage.values());
    }

    public void deleteById(String id) {
        storage.remove(id);
    }

    public void clear() {
        storage.clear();
    }
}
```

Explication détaillée

ConcurrentHashMap au lieu de HashMap : un HashMap classique n'est pas thread-safe — si plusieurs threads écrivent en même temps, les données peuvent se corrompre. ConcurrentHashMap verrouille par petits segments (buckets), donc plusieurs threads peuvent lire/écrire en parallèle sans tout bloquer globalement.

save() : vérifie que l'item et son id ne sont pas null avant de l'ajouter, puis fait `storage.put(id, item)` — un `put()` avec une clé existante remplace automatiquement l'ancien item (upsert).

findById() renvoie un **Optional** : plutôt que de renvoyer null si l'item n'existe pas (risque de `NullPointerException` chez l'appelant), on encapsule le résultat dans un `Optional`. Exemple : `repository.findById("X").orElse(null)` ou `.ifPresent(item -> ...)` forcent l'appelant à gérer explicitement le cas « absent ».

findAll() : renvoie une copie (`new ArrayList<>(storage.values())`) plutôt que la collection interne directement, pour éviter que l'appelant modifie accidentellement le stockage interne.

deleteById() / clear() : opérations CRUD basiques : suppression d'un item par id, ou vidage complet du repository (utile entre deux tests par exemple).

Analyse pédagogique du professeur

Thread safety via ConcurrentHashMap : un HashMap standard n'est pas sûr sous des opérations concurrentes d'un serveur web ; ConcurrentHashMap utilise un verrouillage au niveau des buckets pour offrir une forte concurrence sans blocage global.

Null safety via Optional : renvoyer un Optional force explicitement les appelants à gérer les items manquants, éliminant les bugs potentiels de NullPointerException.